Practical Dense-Key Bootstrapping with Subring Secret Encapsulation

Shihe Ma[1[0009−0006−9212−3260]], Tairong Huang[2], Anyu Wang[2,3,4,5,6[0000−0002−1086−0288](✉)], and Xiaoyun Wang[2,3,4,5,6,7]

[1] Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing, China, msh24@mails.tsinghua.edu.cn
[2] Institute for Advanced Study, Tsinghua University, Beijing, China, huangtr@mail.tsinghua.edu.cn, {anyuwang,xiaoyunwang}@tsinghua.edu.cn
[3] State Key Laboratory of Cryptography and Digital Economy Security, Tsinghua University, Beijing, China
[4] School of Cryptologic Science and Engineering, Shandong University, Jinan, China
[5] Zhongguancun Laboratory, Beijing, China
[6] National Financial Cryptography Research Center, Beijing, China
[7] Shandong Institute of Blockchain, Shandong, China

Abstract. Bootstrapping remains the primary bottleneck in most FHE schemes, significantly impacting their efficiency. To enhance both the speed and precision of bootstrapping, sparse secrets have been widely adopted, particularly in SIMD-style FHE schemes such as BGV, BFV, and CKKS. However, the security of sparse LWE secrets is not well understood, leading to their exclusion from standardization efforts. To address this gap between the potential security risks of sparse secrets and the inefficiency of dense-secret bootstrapping, we introduce the subring secret encapsulation method. This approach involves switching to a dense secret in a subring before bootstrapping, thereby improving bootstrapping performance while still basing security on dense secret LWE. The EvalMod and digit removal steps are accelerated due to the smaller Hamming weight of the subring secret. Furthermore, the algebraic structure of the subring secret enables faster CoeffsToSlots and SlotsToCoeffs operations through hoisted key switchings. When applied to the CKKS scheme, our method achieves a bootstrapping throughput increase of 46%–51% compared to state-of-the-art dense secret bootstrapping techniques. For BGV/BFV schemes, our approach demonstrates a 2.48x improvement in throughput when bootstrapping 2^{15} slots modulo 65537.

Keywords: Fully Homomorphic Encryption · Bootstrapping · Dense secret.

1 Introduction

Fully homomorphic encryption (FHE) enables computation over encrypted data without decrypting it. Currently, most FHE schemes have noisy ciphertexts constructed from NTRU [37], Learning with Errors (LWE) [49], or Ring-LWE

(RLWE) [43] samples. Therefore, they all need Gentry's blueprint of bootstrapping to realize infinite homomorphic computations, which homomorphically decrypts a noisy ciphertext to reset its noise level [29].

For the most widely used FHE schemes, e.g., BFV/BGV [12,11,25], CKKS [19], and FHEW/TFHE [23,21], bootstrapping plays a central role in the construction of the schemes and has undergone extensive studies. However, bootstrapping typically exhibits a time complexity magnitudes larger than homomorphic arithmetic operations and is the efficiency bottleneck in FHE schemes. To enable faster bootstrapping, sparse secrets with bounded Hamming weights have been widely adopted, especially in BFV, BGV, and CKKS [33,17,44]. These schemes usually utilize the sparse secret encapsulation method to switch the dense main secret to an ephemeral sparse secret before bootstrapping, whose Hamming weight can be as low as 32 [10]. However, the security of sparse secrets is not well understood for now, and attacks on sparse secrets are constantly emerging [22,18,46,20,36,39,47]. The lack of confidence in the security guarantees of sparse secrets has led the FHE community to use dense secrets for standardization and industry guidelines [3,8]. Yet, compared to the sparse secret case, FHE bootstrapping with dense secrets suffers from significant performance degradation or limitations on parameter choice. For example, CKKS bootstrapping with dense secrets has lower precision and fewer remaining levels [9,8], and BGV/BFV bootstrapping requires the homomorphic evaluation of polynomials with much higher degrees [44]. The less efficient bootstrapping has become one of the obstacles to the real-world adoption of dense-key FHE schemes.

1.1 Our Contributions

Sparse secrets improve the bootstrapping efficiency, while dense secrets offer stronger security guarantees. To realize the best of the two worlds, we propose the *subring secret encapsulation*, which switches to a subring secret before bootstrapping. This approach leverages the dense-secret RLWE problem within the subring, addressing the security concerns of sparse secrets. Moreover, we demonstrate that subring keys reduce computational costs in the homomorphic rounding, CoeffsToSlots, and SlotsToCoeffs steps in bootstrapping.

Compared to the state of the art in dense-secret CKKS bootstrapping [9,8], our improved bootstrapping shows a 46%∼51% increase in bootstrapping throughput under the same level of failure probability. We also realize the first dense-secret CKKS bootstrapping with a failure probability less than 2^{-128} and a bootstrapping throughput still higher than [9], while the latter has a failure probability of 2^{-15}. The low failure probability is necessary to achieve a high level of IND-CPA$^{\mathrm{D}}$ security, as bootstrapping failures can be exploited to mount key recovery attacks [16].

When applied to dense-secret BGV/BFV bootstrapping, our method shows better efficiency gains under SIMD-friendly parameters. Previous work exploits the high extension degree of the Galois field/ring in each slot to efficiently evaluate large-degree polynomials during bootstrapping [48]. However, as a stronger SIMD property implies a smaller extension degree, its performance degrades

when the number of slots grows. Compared to the state of the art, our method exhibits a 2.48x bootstrapping throughput where the ciphertext encodes 2^{15} slots in $\mathbb{Z}_{65537}$.

In summary, subring secret encapsulation reduces the efficiency gap between dense-key and sparse-key bootstrapping methods by improving the bootstrapping throughput while keeping the dense-secret LWE as the underlying hardness problem. Compared to CKKS bootstrapping with sparse secret encapsulation and a failure probability $< 2^{-128}$, our method is the first to achieve the same level of failure probability under a dense key, exhibiting a 2.44x throughput gap from the sparse secret version. For BGV/BFV bootstrapping with $< 2^{-128}$ failure probability and typical parameters, our method narrows the efficiency gap between dense-key and secret-key bootstrapping from 5.90x down to 2.37x. We believe the reduced gap in bootstrapping efficiency between sparse and dense secrets will promote the adoption of dense secrets, especially in scenarios like FHE standardization, where the potential security risks of sparse secrets cannot be tolerated.

1.2 Technical Overview

Let $R = \mathbb{Z}[X]/(X^N + 1)$, and let $R' = \mathbb{Z}[X']/(X'^{N'} + 1)$ with $X' = X^{N/N'}$ be a subring of R. Subring secret encapsulation means switching from the regular secret $s \in R$ to a uniform ternary secret $s' \in R'$ before bootstrapping, whose security has been proven by Gentry et al. to rely on the dense-key decisional RLWE problem over R' [30]. The most intuitive advantage of s' is that its Hamming weight h is N/N' times smaller than that of a large ring secret. Since the EvalMod step in CKKS evaluates a polynomial approximation of $x \bmod 1$ over $x \in [-K, K]$ with $K = O(\sqrt{h})$, the subring secret with a smaller h reduces the range of the approximation. This leads to a reduction in both computational complexity and the number of levels consumed in EvalMod.

If we consider only its effect on EvalMod, subring secret encapsulation may seem to be the dense key version of sparse secret encapsulation [10]. However, with their special algebraic properties, subring secrets also benefit the evaluation of CoeffsToSlots and SlotsToCoeffs, an advantage not possessed by sparse secrets. The Galois automorphisms in R consist of θ_i for $i \in \mathbb{Z}_{2N}^*$, where θ_i sends X to X^i. The multiplication between a plaintext matrix $M \in \mathbb{C}^{N/2 \times N/2}$ and a CKKS ciphertext c is realized as

$$\sum_{i=0}^{N/2-1} \mathrm{diag}_i(M) \cdot \mathrm{KeySwitch}_{\mathrm{rot}_i(s') \to s'}(\mathrm{rot}_i(c)),$$

where $\mathrm{diag}_i(M) \in R$ stores the i-th diagonal of M and $\mathrm{rot}_i = \theta_{5^i}$. Observe that the automorphisms $\mathrm{rot}_{i+k \cdot N'/2}$ with $k = 0, 1, \ldots, N/N' - 1$ degenerate to the same automorphism on the subring R' because $5^{i+k \cdot N'/2} \equiv 5^i \bmod 2N'$ [30]. Thus, we can 'hoist' the KeySwitch operations whose rotation offsets are con-

gruent modulo $N'/2$, leading to the improved linear transformation

$$\sum_{i=0}^{N'/2-1} \text{KeySwitch}_{\text{rot}_i(s')\to s'}\Big(\sum_{j=0}^{N/N'-1} \text{diag}_{i+jN'/2}(M)\cdot \text{rot}_{i+jN'/2}(c)\Big). \quad (1)$$

The linear transformation has a lower complexity cost due to fewer calls to KeySwitch. The hoisting of KeySwitch is also compatible with BSGS matrix multiplication [31], whose details will be described in later sections.

We have seen that general linear transformations in CKKS benefit from a subring secret. The improvement can be further enhanced for CoeffsToSlots and SlotsToCoeffs using their FFT-like factorization. Following known results [13,34], CoeffsToSlots involves the homomorphic left multiplication by

$$S_{N/2}^{-1} = E_{N/2}^{-1}\cdot E_{N/4}^{-1}\cdots E_2^{-1},$$

where E_k^{-1} is realized with homomorphic automorphisms of $\text{rot}_{\pm N/2k}$. Under the subring secret s', the automorphisms used in E_k^{-1} fix s' as long as $k \le N/N'$, meaning that $E_{N/N'}^{-1}\cdots E_4^{-1}\cdot E_2^{-1}$ can be computed without key switching. This allows us to evaluate these matrices with little running time and no level consumption, leading to improved CoeffsToSlots. SlotsToCoeffs in slim bootstrapping evaluates $S_{N/2}$ homomorphically and can also be improved. By switching the secret to s' in the middle of SlotsToCoeffs, the remaining matrices $E_2\cdot E_4\cdots E_k$ could be evaluated with hoisted key switching as in Equation 1. A diagram of the modified slim bootstrapping is detailed in Figure 1[1].

BGV/BFV bootstrapping benefits from subring secret encapsulation similarly to CKKS. The major differences are that (1) the running time of homomorphic digit removal is more closely related to the secret Hamming weight than EvalMod in CKKS [44], making the efficiency gain of the subring secret over the dense main secret more significant, and (2) the homomorphic linear transformations depend on the plaintext space parameters and require Frobenius automorphisms [32]. The details can be found in Section 4.

1.3 Related Works

Closely related works on FHE bootstrapping [9,48,10,8] have been discussed earlier, while other related works are given here.

Ring switching [30]. The idea of switching the secret of an RLWE ciphertext over R_q to a secret $s' \in R'$ was first adopted in the ring switching method by Gentry et al. After the key switching, the ciphertext can be broken into multiple RLWE ciphertexts over R'_q encrypted under s' through a R'_q-module isomorphism. Combined with proper homomorphic linear transformations, the ciphertexts over R'_q will eventually encrypt the slot values of the original ciphertext,

[1] The coefficients in slots are actually permuted due to the bit reversal property of FFT. The diagram does not show this for simplicity.

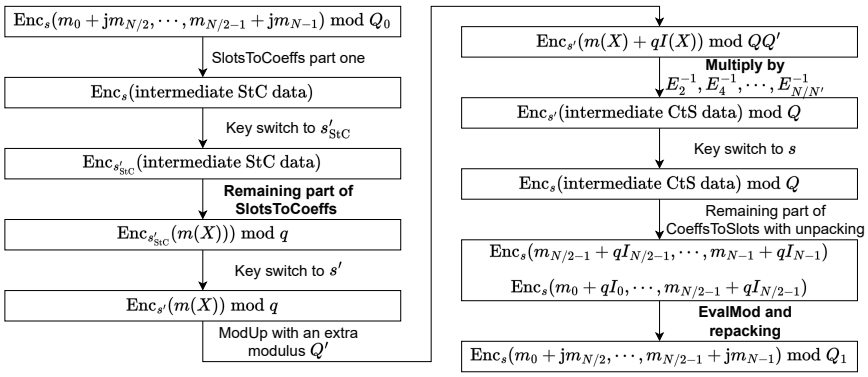

Fig. 1. The modified slim bootstrapping procedure. Steps that benefit from using a subring secret are highlighted in bold. Q is the ciphertext modulus after ModUp in a regular bootstrapping. Q' should provide enough levels for the key-switching-free linear transformations. The functionality of bootstrapping is ensured by $Q_1 \gg Q_0$.

realizing ring switching. Although ring switching could be used to accelerate bootstrapping by reducing the ciphertext size [12], for SIMD packed ciphertexts (which is usually the case in FHE nowadays), it would involve unpacking the ciphertext into low-dimensional sub-ciphertexts, applying a multi-ciphertext linear transformation on them to recover the slot values, performing bootstrapping for each sub-ciphertext, and finally repacking the ciphertexts. In contrast, our method only needs a key-switching to the subring secret, which is technically simpler and computationally easier.

HERMES for ring packing [6]. HERMES also packs multiple RLWE ciphertexts into one ciphertext using ring switching before performing CKKS bootstrapping. However, they switched to a sparse secret before CKKS bootstrapping and did not exploit the properties of the dense subring secret to improve bootstrapping efficiency. Thus, their and our works focus on different aspects.

Extended TFHE bootstrapping [41]. By embedding the test vector and bootstrapping keys in R into a larger cyclotomic ring $\bar{R} > R$, the extended TFHE bootstrapping (EBS) enjoys the lower discretization error in $\bar{R}$ while keeping the noise growth of R. Their and our works apply to different FHE schemes and are not closely relevant.

1.4 Paper Organization

Section 2 introduces the notations and basics on CKKS and BGV/BFV schemes. Section 3 describes the application of subring secret encapsulation to CKKS in detail, including its effect on EvalMod, CoeffsToSlots/SlotsToCoeffs, and

sparsely packed ciphertexts, while the impact of subring secret encapsulation on BGV/BFV bootstrapping is given in Section 4. Section 5 presents the selected parameters and implementation results.

2 Preliminaries

In this section, we introduce some background knowledge that we will need in later sections.

2.1 Basic Notations

We identify $\mathbb{Z}_q$ as the set of integers in $[-\frac{q}{2}, \frac{q}{2})$ for $q > 2$ and $\mathbb{Z}_2$ as $\{0, 1\}$. For $a \in \mathbb{Z}$ and a positive integer q, $[a]_q := a \bmod q \in \mathbb{Z}_q$. The M-th primitive root of unity is denoted as ζ_M. In the complex field $\mathbb{C}$, $\zeta_M = e^{2\pi i/M}$. For a length n vector a, $a \lll k$ means cyclic rotating a by k positions to its left. Logarithms are base two by default.

For power-of-two integers N, N' where $N'|N$, the cyclotomic rings of the corresponding degree are denoted $R = \mathbb{Z}[X]/(X^N + 1)$ and $R' = \mathbb{Z}[X']/(X'^{N'} + 1)$. R' is identified as a subring of R through $X' = X^{N/N'}$. We let $R_q = R/qR$ for an integer q. For $i \in \mathbb{Z}_{2N}^*$, $\theta_i(m(X)) = m(X^i)$ for $m(X) \in R$ form the automorphism group of R.

Ring Learning with Errors (RLWE) [43]. RLWE cryptosystems over R_q sample a secret key $s \in R$ with a small bound. A ciphertext encrypting $m \in R$ has the form of $(a, b) = (a, as + m + e) \in R_q^2$, where $a \leftarrow R_q$ and the error $e \in R$ is small. To decrypt, one calculates $-as + b = m + e$. For redundantly encoded m, m could be recovered from $m + e$ by rounding. The security of the cryptosystem is based on the hardness of the decisional RLWE problem, which is distinguishing RLWE ciphertexts of 0 from the uniform distribution in R_q^2.

Gadget decomposition. Gadget decomposition is parameterized by a modulus q and a gadget vector $\boldsymbol{g} \in \mathbb{Z}^D$. Given $a \in R_q$, its gadget decomposition with respect to $\boldsymbol{g}$ is defined as $\boldsymbol{g}^{-1}(a) = (a_0, a_1, \cdots, a_{D-1}) \in R_q^D$, satisfying $\langle \boldsymbol{g}^{-1}(a), \boldsymbol{g} \rangle \equiv a \bmod q$. a_i has the desired property that $|a_i|_\infty < B_i \approx q^{1/D}$. Thus, gadget decomposing a R_q element before multiplying it with ciphertexts (usually key materials) helps to reduce the noise growth. Common choices for the gadget vector include the radix vector $g_i = b^i$ for some base b, the RNS vector or a mixture of both [7].

2.2 CKKS Scheme

Plaintext encoding. CKKS encodes a plaintext vector in $\mathbb{C}^{N/2}$ into a polynomial in $m \in R$, where each entry in the plaintext vector is called a slot. Homomorphic addition and multiplication in R apply to the plaintext vector slotwise. This is known as the SIMD (single instruction multiple data) property.

The slot vector $v \in \mathbb{C}^{N/2}$ is encoded in $m \in R$ through the inverse Discrete Fourier Transform over $\zeta_{2N}^{5^i}$ for $i = 0, 1, ..., \frac{N}{2} - 1$. During the encoding, a large scaling factor Δ is multiplied with v to reduce the precision loss from rounding.

- $\mathrm{Ecd}(v \in \mathbb{C}^{N/2}, \Delta \in \mathbb{R}) = \lfloor \Delta \cdot \mathrm{DFT}^{-1}(v) \rceil \in R$
- $\mathrm{Dcd}(m \in R, \Delta \in \mathbb{R}) = \mathrm{DFT}(m)/\Delta \in \mathbb{C}^{N/2}$

The cyclic rotation and elementwise conjugation of the slot vector is realized by $\mathrm{rot}_i = \theta_{5^i}$ and $\mathrm{conj} = \theta_{-1}$, satisfying $\mathrm{Dcd}(\mathrm{rot}_i(m), \Delta) = \mathrm{Dcd}(m, \Delta) \lll i$ and $\mathrm{Dcd}(\mathrm{conj}(m), \Delta) = \overline{\mathrm{Dcd}(m, \Delta)}$. We abuse the notation and let rot_i and conj also apply to ciphertexts, e.g., $\mathrm{rot}_i((a, b)) = (\mathrm{rot}_i(a), \mathrm{rot}_i(b))$.

A ciphertext is called sparsely packed if the slot vector v consists of N/N^* concatenations of the same $v^* \in \mathbb{C}^{N^*/2}$. In this case, $\mathrm{Ecd}(v) \in R^*$, where R^* is a subring of R with a dimension of N^*.

Scheme description. The basic algorithms for CKKS are given below. Some details are omitted because they are irrelevant to this paper.

- **Keygen**$(1^\lambda, N, \chi_s, \chi_e)$: Given a security parameter λ, a ring dimension N, the secret and noise distributions χ_s and χ_e, return a secret $s \leftarrow \chi_s$, the maximum RLWE modulus Q.
- **Enc**(m, s, q): Given a plaintext polynomial m, a secret s and a ciphertext modulus q, return an RLWE ciphertext encrypting m under s as $c = (a, b) = (a, as + m + e) \in R_q^2$. We use $\mathrm{Enc}_s(m)$ to represent the set of ciphertext encrypting polynomials close enough to m.
- **Dec**$(c \in R_q^2, s)$: Given a ciphertext c, the secret s, return $[\langle c, (-s, 1)\rangle]_q$ as the decryption result.
- **KSKgen**$(s_0, s_1, q, P, \boldsymbol{g})$: Given the source and target secrets s_0 and s_1, a ciphertext modulus q, an auxiliary modulus P and a gadget vector $\boldsymbol{g} \in \mathbb{Z}^D$, return $\mathrm{KSK}_{s_0 \to s_1} = (\mathrm{Enc}_{s_1}(Pg_i s_0))_i \in R_{qP}^{D \times 2}$, which serves as the key-switching key from s_0 to s_1.
- **Rescale**$(c \in R_q^2, q^* | q)$: Given a ciphertext c modulo q and another modulus $q^* | q$, return $c^* \in R_{q^*}^2$ with $\mathrm{Dec}(c^*, s) \approx \mathrm{Dec}(c, s) \cdot \frac{q^*}{q}$.
- **KeySwitch**$(c = (a, b) \in \mathrm{Enc}_s(m), \mathrm{KSK}_{s \to s^*})$: Given a ciphertext c modulo q and $\mathrm{KSK}_{s \to s^*}$, return $c^* \in \mathrm{Enc}_{s^*}(m)$, which is calculated as $c^* = (0, b) - \mathrm{Rescale}(\langle \boldsymbol{g}^{-1}(a \bmod qP), \mathrm{KSK}_{s \to s^*} \rangle, q)$.
- **HomAdd**(c_0, c_1): Given two ciphertexts c_0, c_1 under the same secret, return $c_0 + c_1 \in R_q^2$.
- **HomPMult**$(c, m \in R)$: Given a ciphertext c and a polynomial m, return $m \cdot c$.
- **HomCMult**$(c_0 \in \mathrm{Enc}_s(m_0), c_1 \in \mathrm{Enc}_s(m_1), \mathrm{KSK}_{s^2 \to s})$: Given two ciphertexts c_0, c_1 under the same secret and the relinearization key $\mathrm{KSK}_{s^2 \to s}$, return $c^* \in \mathrm{Enc}_s(m_0 \cdot m_1)$.
- **HomRot**$_i(c \in \mathrm{Enc}_s(\mathrm{Ecd}(v, \Delta)), \mathrm{KSK}_{\mathrm{rot}_i(s) \to s})$: Given a ciphertext c and the rotation key with offset i, return $c^* \in \mathrm{Enc}_s(\mathrm{Ecd}(v \lll i, \Delta))$.
- **HomConj**$(c \in \mathrm{Enc}_s(\mathrm{Ecd}(v, \Delta)), \mathrm{KSK}_{\mathrm{conj}(s) \to s})$: Given a ciphertext c and the conjugation key, return $c^* \in \mathrm{Enc}_s(\mathrm{Ecd}(\bar{v}, \Delta))$.

2.3 BGV and BFV Schemes

Plaintext encoding. The plaintext space of BGV/BFV is R_{p^r}, where the prime power p^r is the plaintext modulus coprime with N. Let $d = \mathrm{ord}_{\mathbb{Z}_{M^*}}(p)$ and $l = N/d$, $R_{p^r} \cong E^l$, where E is a Galois field or Galois ring with characteristic p^r and $[E : \mathbb{Z}_{p^r}] = d$. Let Ecd be the map from E^l to R_{p^r}, and let $\mathrm{Dcd} = \mathrm{Ecd}^{-1}$.

A ciphertext is called sparsely packed if the vector of slot values v resides in $\mathbb{Z}_{p^r}^l \subset E^l$. In this case, $\mathrm{Ecd}(v) \in R^*$, where R^* is a subring of R with a dimension of $N^* = N/d$ for $p \equiv 1 \bmod 4$, or $N^* = 2N/d$ for $p \equiv 3 \bmod 4$.

Slot arrangement and linear transformation. With N a power of two, $H = \mathbb{Z}_M^*/\langle p \rangle$ is generated by $g_1 = 5, g_2 = -1$ when $p \equiv 1 \bmod 4$, and by $g_1 = 5$ when $p \equiv 3 \bmod 4$. Let $l_i = \mathrm{ord}_H(g_i)$, the slots are arranged as a hypercube with a shape of $l_1 \times l_2 \times \cdots$. If $l_i = \mathrm{ord}_{\mathbb{Z}_M^*}(g_i)$, the i-th dimension is called a good one and $\mathrm{rot}_{i,j} = \theta_{g_i^j}$ is a cyclic rotation by j positions on every hypercolumn along the i-th dimension. Otherwise, the dimension is called a bad one and $\mathrm{rot}_{i,j}(m) = \theta_{g_i^j}(m) \cdot \mu_{i,j} + \theta_{g_i^{j-l_i}}(m) \cdot (1 - \mu_{i,j})$ for some constant $\mu_{i,j} \in R_{p^r}$. The first dimension is good when $d = 1$ and $p \equiv 1 \bmod 4$, or $d = 2$ and $p \equiv 3 \bmod 4$. The second dimension is always good for $p \equiv 1 \bmod 4$. $\sigma(m(X)) = m(X^p)$ acts as the Frobenius automorphism on the slots. We extend the notations above to the subring $R' < R$ by adding a prime over them, i.e., E', l', d', etc.

Homomorphic operations. The basic algorithms for BGV/BFV are given below.

- $\underline{\mathbf{Keygen}(1^\lambda, N, \chi_s)}$: Given a security parameter λ, a ring dimension N and a secret distribution χ_s, return a secret $s \leftarrow \chi_s$, the maximum RLWE modulus Q.
- $\underline{\mathbf{Enc}(m \in R_{p^r}, s, q)}$: Given a plaintext polynomial m, a secret s and a ciphertext modulus q, sample an RLWE instance $(a, b) \in R_q^2$ with secret s. If the scheme is BGV, q should be coprime with p, and the returned ciphertext is $(p^r a, p^r b + m)$. For the BFV scheme, return $(a, b + \lfloor \frac{p^r}{q} m \rceil)$. We use $\mathrm{Enc}_s(m)$ to represent the set of ciphertexts encrypting the polynomial m.
- $\underline{\mathbf{Dec}(c \in R_q^2, s, p^r)}$: Given a ciphertext c, the secret s, return the decrypted polynomial. For the BFV scheme, return $[\frac{p^r}{q}(\langle c, (-s, 1) \rangle)]_{p^r}$. For the BGV scheme, return $[[\langle c, (-s, 1) \rangle]_q]_{p^r}$.
- $\underline{\mathbf{ModSwitch}(c \in R_q^2, q^*|q)}$: Given a ciphertext c modulo q and a target modulus $q^*|q$, return $c' \in R_{q^*}^2$ with $\mathrm{Dec}(c^*, s) = \mathrm{Dec}(c, s)$.
- $\underline{\mathbf{KSKgen}(s_0, s_1, q, P, \boldsymbol{g})}$: this is similar to the CKKS case, but the key materials are encrypted in the CKKS style for BFV and in the BGV style for BGV.
- $\underline{\mathbf{KeySwitch}(c \in \mathrm{Enc}_s(m), \mathrm{KSK}_{s \to s^*})}$: Given a ciphertext c modulo q and $\mathrm{KSK}_{s \to s^*}$, return $c^* \in \mathrm{Enc}_{s^*}(m)$, which is calculated as $c^* = (0, b) - \mathrm{ModSwitch}(\langle \boldsymbol{g}^{-1}(a \bmod qP), \mathrm{KSK}_{s \to s^*} \rangle, q)$.
- $\underline{\mathbf{HomAdd}(c_0, c_1)}$: identical to the CKKS case.
- $\underline{\mathbf{HomPMult}(c, m \in R_{p^r})}$: identical to the CKKS case.

- **HomCMult**$(c_0 \in \mathrm{Enc}_s(m_0), c_1 \in \mathrm{Enc}_s(m_1), \mathrm{KSK}_{s^2 \to s})$: Given two ciphertexts c_0, c_1 under the same secret and a relinearization key $\mathrm{KSK}_{s^2 \to s}$, return $c^* \in \mathrm{Enc}_s(m_0 \cdot m_1)$.
- **HomRot**$_{i,j}(c \in \mathrm{Enc}_s(m), \mathrm{KSK}_{\mathrm{rot}_{i,j}(s) \to s})$: Given a ciphertext c and the rotation key with offset j in dimension i, return $c^* \in \mathrm{Enc}_s(\mathrm{rot}_{i,j}(m))$.
- **HomFrob**$(c \in \mathrm{Enc}_s(m), \mathrm{KSK}_{\sigma(s) \to s})$: Given a ciphertext c and the Frobenius key, return $c^* \in \mathrm{Enc}_s(\sigma(m))$.

2.4 Bootstrapping

CKKS and BGV/BFV follow the same bootstrapping framework of (1) increasing homomorphic capacity, (2) CoeffsToSlots (CtS), (3) homomorphic rounding, and (4) SlotsToCoeffs (StC).

- Step 1 occurs at a ciphertext modulus of q_0, the lowest modulus that is large enough for the correct decryption. It increases the homomorphic capacity of the ciphertext while introducing noise into the underlying plaintext. In the case of CKKS, step 1 is called Modup, which directly raises the ciphertext modulus from q_0 to a large Q, turning the encrypted plaintext polynomial m into $m + q_0 I$. For BGV/BFV, this step turns $m \in R_{p^r}$ into $p^t m + I \in R_{p^{r+t}}$, where $|I| < p^t/2$ for correctness.
- CtS moves the coefficients of the noisy plaintext polynomial into the slots by homomorphically evaluating Ecd on the slots. This ensures the next step can exploit the SIMD feature of the slot encoding for better efficiency.
- Step 3 removes the noise introduced into the plaintext space by step 1 by homomorphically evaluating some polynomials. In CKKS bootstrapping, this step evaluates an approximation of the modular reduction function on two ciphertexts to obtain $m_i = (m_i + q_0 I_i) \bmod q_0$ in each slot and is called EvalMod. In BGV/BFV bootstrapping, it removes the lowest t p-ary digits in $p^t m_i + e_i \in \mathbb{Z}_{p^{r+t}}$ to get $m_i \in \mathbb{Z}_{p^r}$ in the slots of d ciphertexts and is called digit removal.
- StC moves the slot values into coefficients of the plaintext polynomial by homomorphically evaluating Dcd on the slots.

If the plaintext is sparsely packed with the plaintext polynomial in $R^* \leq R$, an extra step called SubSum is required before CoeffsToSlots. SubSum removes unwanted coefficients of the noisy plaintext polynomial by evaluating Tr_{R/R^*} homomorphically [17]. For sparsely packed BGV/BFV/CKKS ciphertexts, digit removal (or EvalMod) operates on only 1 instead of d (or 2) ciphertexts, thus leading to a shorter running time.

Apart from the ModUp-first bootstrapping, the bootstrapping steps can be reordered as StC, increasing homomorphic capacity, CtS, and homomorphic rounding. This way of bootstrapping is known as the slim bootstrapping [14] or StC-first bootstrapping [5]. Slim bootstrapping decreases the computational cost of StC because it is performed with fewer RNS primes. Despite a shorter bootstrapping time, slim bootstrapping means that regular homomorphic computation is performed at a higher modulus and consequently slower. Thus, whether

slim bootstrapping could decrease the total running time depends on the cost of the computation task executed.

2.5 Ring switching

Low-level ciphertexts in a homomorphic circuit have a lower modulus-to-noise ratio than the freshly encrypted ones, meaning a lower RLWE dimension suffices to provide the same noise and security level. Gentry et al. exploited this fact to break a low-level ciphertext into multiple ciphertexts in a subring of the current cyclotomic ring, and this technique is called *ring switching* [30].

Recall R and R' are cyclotomic rings of dimension N and N', respectively. For a SIMD-packed ciphertext c over R_q, ring-switching it into R'_q consists of three steps[2].

1. Perform key-switching on c to obtain c', an encryption of the same polynomial under another secret $s' \in R^{(M')}$.
2. Use a map $T : R_q \to R'^{N/N'}_q$ to break c' into N/N' ciphertexts over R', denoted as c'_i, $i = 0, 1, \cdots, N/N' - 1$.
3. Evaluate a linear transformation over all c_i's to recover the slot values of c inside the slots of c_i's.

Here the map $T : R_q \to R'^{N/N'}_q$ is defined as $T(a) = (a_0, a_1, ..., a_{N/N'-1})$ with $a_i \in R'_q$ and $a = \sum_{i=0}^{N/N'-1} a_i X^i$, which is a R'_q-module isomorphism.

The security of subring key-switching is based on the RLWE problem over R'_q, as detailed in the following lemma.

Lemma 1. [3] *[30] If the decisional RLWE problem over R'_q with error distribution χ'_e is hard, then so is the decisional RLWE problem over R_q with error distribution $\chi_e = T^{-1}((\chi'_e)^{N/N'})$, but where the secret is chosen from χ'_s, and in particular is in subring R'_q.*

Intuitively, applying T on a ciphertext over R_q with secret $s' \in R'$ will decompose it into N/N' ciphertexts over R'_q under s', leading to the proof of the lemma. Also, by taking both χ_e and χ'_e as coefficient-wise Gaussian distributions with the same standard deviation, the subring key-switching material is exactly $\mathrm{KSK}_{s \to s'}$, whose generation has been described in **KSKgen**$(\cdots)$ in previous subsections.

Although a small N' is preferable for better performance, it cannot be arbitrarily small. This is because the maximum allowed ciphertext modulus q decreases with N' to meet a target security level, while q needs to be large enough so that the ciphertext after subring key switching still encrypts the original message with a predefined level of precision. Thus, the concrete choice of N' depends on the message precision, i.e., the scaling factor of CKKS ciphertexts and the plaintext modulus of BGV/BFV ciphertexts. Table 1 is taken from the FHE guideline [8] and gives the maximum values of q for different ring dimensions.

[2] We are only interested in power-of-two cyclotomic rings here, although the original ring switching applies to arbitrary cyclotomic rings.

[3] $\chi'_s = \chi'_e$ in the original paper, but the lemma is still valid for distinct χ'_s and χ'_e.

Table 1. Maximum values of q with a uniform ternary secret, a Gaussian error with standard deviation 3.19, and 128-bit security [8].

N'	2^{10}	2^{11}	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}
$\log_2 q$	26	53	106	214	430	868	1747

3 Subring Secret Encapsulation for CKKS Bootstrapping

Our main observation can be summarized as this: by switching the regular secret to a subring secret (as done in the first step of a ring switching), we could improve the efficiency of CKKS bootstrapping while still providing a security guarantee of a dense key, according to Lemma 1. Notably, our method does not perform the other two steps in a ring-switching, thus more similar to a subring secret version of the sparse secret encapsulation [10].

3.1 EvalMod with a Subring Secret

In FHE bootstrapping, the homomorphic rounding step (EvalMod in CKKS and digit removal in BGV/BFV) usually dominates the running time, especially for dense secrets. Let K denote the upper bound on the error to be eliminated. It is established that K asymptotically equals $O(\sqrt{h})$ [17], where h represents the Hamming weight of the secret key during the homomorphic capacity increasing step. With other parameters fixed, the computational cost and precision loss of homomorphic rounding grow with K.

Key-switching to a subring secret just before performing homomorphic capacity increase has the same benefit as the sparse secret encapsulation: the lower Hamming weight of the subring secret leads to a smaller K, thus improving the efficiency of the homomorphic rounding stage. A bound on K is estimated in Lemma 2.

Lemma 2. *When encapsulating a subring secret with dimension N', the noise polynomial I that needs to be removed during bootstrapping satisfies*

$$\Pr[||I||_\infty > K] \leq N \cdot \operatorname{erfc}\left(\frac{K}{\sqrt{2(\frac{2}{3}N' + k \cdot \sqrt{\frac{2}{9}N'} + 1)/12}}\right)$$

with a failure rate of $\operatorname{erfc}(k/\sqrt{2})/2$.

Proof. We extend the framework of [8] to compute estimates for K, incorporating minor adaptations to simplify the estimation. Since h follows the binomial distribution with parameters N' and $p = \frac{2}{3}$, h can be approximated as a Gaussian variable with standard deviation $\sigma_h = \sqrt{\frac{2N'}{9}}$ centered at $E[h] = \frac{2N'}{3}$. We use reject sampling to sample a secret with a Hamming weight of $\bar{h} \leq E[h] + k \cdot \sigma_h$, where k is chosen to ensure $\Pr[h > E[h] + k\sigma_h] = \Pr[|h - E[h]| > k\sigma_h]/2 = \operatorname{erfc}(k/\sqrt{2})/2 < 2^{-128}$.

Since each coefficient of I is the sum of at most $\bar{h}+1$ random variables sampled from $U(-0.5, 0.5)$, it is modeled as a centered Gaussian variable with a standard deviation of $\sigma_I = ((\bar{h}+1)/12)^{1/2}$. The union bound can be adopted to evaluate the probability

$$\Pr[||I||_\infty > K] \leq N \cdot \mathrm{erfc}(\frac{K}{\sqrt{2} \cdot \sigma_I}).$$

□

Compared to the standard bootstrapping with a dense secret, the subring encapsulation reduces the value of K by a factor of about $\frac{N}{N'}$, according to Lemma 2. For a better understanding of the benefits of a smaller K, We briefly review the existing methods for EvalMod.

Existing methods for EvalMod. The input ciphertext for EvalMod stores $\frac{\Delta}{q_0} m_i + I_i \in \bigcup_{i=-K+1}^{K-1} (i - \frac{\Delta}{q_0}, i + \frac{\Delta}{q_0}) \subset (-K, K)$ in its slots. The EvalMod step in CKKS bootstrapping homomorphically reduces the slot values modulo 1 and scales the output by $\frac{q_0}{\Delta}$ to obtain m_i in slots. Most realizations of EvalMod need to approximate some non-polynomial function with a polynomial over $(-K, K)$ or its subintervals where $\frac{\Delta}{q_0} m_i + I_i$ lies. The approximated function can be the exponential function $e^{2\pi i x}$ [17,38], the sine or cosine function [13,35,40], or the modular reduction function $x \bmod 1$ [40,42]

In the example of realizing EvalMod with the cosine function, let $t = \frac{\Delta}{q_0} m_i + I_i$, the function $\frac{1}{2\pi} \cos(2\pi(t - \frac{1}{4}))$ serves as a good approximation to $f(t) = t \bmod 1$ over $t \in \bigcup_{i=-K+1}^{K-1} (i - \frac{\Delta}{q_0}, i + \frac{\Delta}{q_0})$, given $\frac{q_0}{\Delta}$ is sufficiently small. The double angle formula $\cos(\theta) = 2\cos(\frac{\theta}{2})^2 - 1$ is applied r times to reduce the problem of approximating $\cos(x) = \cos(2\pi(t - \frac{1}{4}))$ over $x \in (2\pi(-K - 1/4), 2\pi(K - 1/4))$ to approximating $\cos(x') = \cos(2^{-r} 2\pi(t - \frac{1}{4}))$ over $x' \in (2^{-r} 2\pi(-K - 1/4), 2^{-r} 2\pi(K - 1/4))$. Finally, the domain of x' is scaled onto $(-1, 1)$ and the scaled cosine function is approximated on the Chebyshev basis $T_n(\cos\theta) = \cos(n\theta)$.

The minimum degree of the approximation polynomial needs to increase with K to achieve a fixed level of approximation error. Using the example, the error of a degree-n minimax (or Chebyshev) approximation of $\cos(x'' \cdot 2^{-r} 2\pi K)$ over $x'' \in (-1, 1)$ is bounded by $\frac{1}{2^n (n+1)!} \cdot (2^{-r} 2\pi K)^{n+1}$ [24]. In addition, the high-precision Chebyshev approximation with optimized nodes proposed by Han and Ki requires a polynomial degree of at least $2K - 2$ [35]. To avoid the costly homomorphic computation of high-degree polynomials, this optimized Chebyshev approximation is adopted only for small values of K.

Summarizing subring secret encapsulation's benefits for EvalMod. As discussed above, obtaining a smaller K through subring secret encapsulation could enable us to (1) use Han and Ki's Chebyshev approximation method for better bootstrapping precision than standard Chebyshev approximation, and (2) use fewer levels of the double angle formula or an approximation polynomial with a lower degree to reduce the number of consumed levels and/or the running time of EvalMod, while maintaining the same bootstrapping precision.

Remark. The theoretical time complexity of EvalMod under the subring secret is difficult to determine. The main obstacle is theoretically deciding the required polynomial degree and iterations of the double-angle formula to reach a given precision, due to the absence of a precise L1 noise analysis for EvalMod in the literature.

3.2 Linear Transformations with a Subring Secret

The usage of the subring secret also accelerates the homomorphic linear transformations. In the ring R, the $\mathbb{Z}$-linear transformations are generated by scalar multiplications and the group of automorphisms $G = \{\theta_i | i \in \mathbb{Z}_{2N}^*\}$. In the subring R', the automorphism group is $G' = \{\theta'_i | i \in \mathbb{Z}_{2N'}^*\}$. Gentry et al. observed that $f(\theta_i) = \theta'_{i \bmod 2N'}$ is an N/N'-to-one homomorphism from G to G' with $\ker f = \{\theta_i | i \equiv 1 \bmod 2N'\}$ [30]. Thus, when the ciphertext is encrypted under a subring secret s', the ring automorphisms in R sharing the same remainder modulo $2N'$ degenerate to the same automorphism on s'. These automorphisms share the key-switching key $\mathrm{KSK}_{\theta(s')\to s'}$, so we can 'hoist' the corresponding key-switchings into a single one when computing homomorphic linear transformations, leading to faster homomorphic computation. We will show that this hoisting property is well compatible with the linear transformations in bootstrapping and helps to reduce the number of consumed levels, running time, and the number of automorphism keys for CoeffsToSlots and SlotsToCoeffs.

Existing methods for homomorphic linear transformations in CKKS. Let $M \in \mathbb{C}^{N/2\times N/2}$ be the matrix representing the linear transformation, and $\mathrm{diag}_i(M) = \mathrm{Ecd}((M_{0,i}, M_{1,i+1}, ..., M_{N/2-1,i-1}), \Delta_0)$ be the polynomial encoding i-th diagonal of M. The homomorphic matrix multiplication of M with $c \in \mathrm{Enc}_s(\mathrm{Ecd}(v, \Delta))$ is realized by

$$\sum_{i=0}^{N/2-1} \mathrm{diag}_i(M) \cdot \mathrm{HomRot}_i(c) \in \mathrm{Enc}_s(\mathrm{Ecd}(M \cdot v, \Delta_0\Delta)).$$

Using the Baby-Step-Giant-Step algorithm (BSGS) with giant step g, we rewrite the rotation index as $i = jg + k$ with $k = i \bmod g$, the equation above can be transformed into

$$\sum_{j\in[\lceil N/(2g)\rceil]} \mathrm{HomRot}_{jg}\Big(\sum_{k\in[g]} \mathrm{rot}_{-jg}(\mathrm{diag}_{jg+k}(M)) \cdot \mathrm{HomRot}_k(c)\Big).$$

Setting $g = O(\sqrt{N})$, BSGS matrix multiplication needs only $O(\sqrt{N})$ rotation keys and $O(\sqrt{N})$ key switchings instead of $O(N)$ in the equation above. With hoisting [32] and its improved version, called double hoisting [9], the number of key switchings in baby step computations can be reduced to one.

Linear transformations under a subring secret $s' \in R'$. Since $\mathbb{Z}_{2N}^* = \langle 5, -1 \rangle$, $\mathrm{rot}_1 = \theta_5$ and $\mathrm{conj} = \theta_{-1}$, the automorphism group $G = \langle \mathrm{rot}_1, \mathrm{conj} \rangle$. Moreover, $\ker f = \langle \mathrm{rot}_{N'/2} \rangle$ because the order of $f(\mathrm{rot}_1)$ is $N'/2$ in G'. This means that homomorphic rotations with the same offset modulo $N'/2$ share the same key-switching key. Let $c' \in \mathrm{Enc}_{s'}(\mathrm{Ecd}(v, \Delta))$, $\mathrm{HomRot}_i(c) = \mathrm{KeySwitch}(\mathrm{rot}_i(c), \mathrm{KSK}_{\mathrm{rot}_{i \bmod N'/2}(s') \to s'}) \in \mathrm{Enc}_{s'}(\mathrm{Ecd}(v \lll i, \Delta))$.

By hoisting the key switchings, the homomorphic linear transformation between M and c' simplifies to

$$\begin{aligned} &\sum_{i=0}^{N/2-1} \mathrm{diag}_i(M) \cdot \mathrm{HomRot}_i(c) \\ = &\sum_{i=0}^{N'/2-1} \mathrm{KeySwitch}\Big(\sum_{j=0}^{N/N'-1} \mathrm{diag}_{i+jN'/2}(M) \cdot \mathrm{rot}_{i+jN'/2}(c), \mathrm{KSK}_{\mathrm{rot}_i(s') \to s'} \Big). \end{aligned}$$

Similar results hold for BSGS matrix multiplications. For simplicity, we assume that the giant step g is a power of two. If $g < N'/2$, the giant step rotations HomRot_{jg} can be hoisted as $\frac{N'}{2g} - 1$ key switchings by rewriting $j = \frac{N'}{2g}u + v$

$$\sum_{v \in [N'/(2g)]} \mathrm{KeySwitch}\Big(\sum_{u \in [N/N']} \mathrm{rot}_{jg}(f_g(j), \mathrm{KSK}_{\mathrm{rot}_{vg}(s') \to s'} \Big),$$

where $f_g(j) = \sum_{k \in [g]} \mathrm{rot}_{-jg}(\mathrm{diag}_{jg+k}(M)) \cdot \mathrm{HomRot}_k(c))$ is the input to HomRot_{jg} in the original BSGS formula.

If $g \geq N'/2$, all HomRot_{jg} can be replaced by rot_{jg} and the baby step rotations HomRot_k can be hoisted by rewriting $k = \frac{N'}{2}u + v$

$$\sum_{j \in [N/(2g)]} \mathrm{rot}_{jg}\Big(\sum_{v \in [N'/2]} \mathrm{KeySwitch}\Big(\sum_{u \in [2g/N']} \mathrm{rot}_{-jg}(\mathrm{diag}_{jg+k}(M)) \cdot \mathrm{rot}_k(c), \mathrm{KSK}_{\mathrm{rot}_v(s') \to s'} \Big) \Big)$$

CoeffsToSlots. The CoeffsToSlots step in CKKS bootstrapping evaluates Ecd on the slots, which is similar to an inverse complex DFT with bit reversal [13,34]. Define $\mathrm{rev}(i)$ as the bit reversal of i, where the bit length of i should be clear in the context. Define $S_n = (\zeta_{4n}^{5^i \cdot \mathrm{rev}(j)})_{0 \leq i,j < n}$. The computationally expensive part of CoeffsToSlots (or SlotsToCoeffs) involves a homomorphically left multiplication of the slot vector by $S_{N/2}^{-1}$ (or $S_{N/2}$). S_n can be recursively factorized as

$$S_n = \begin{pmatrix} I_{n/2} & W_{n/2} \\ I_{n/2} & -W_{n/2} \end{pmatrix} \cdot \begin{pmatrix} S_{n/2} & 0 \\ 0 & S_{n/2} \end{pmatrix},$$

where $W_{n/2} = \mathrm{diag}(\zeta_{4n}^{5^i})_{0 \leq i < n/2}$. Applying the factorization above to $S_{N/2}^{-1}$ results in

$$S_{N/2}^{-1} = E_{N/2}^{-1} \cdot E_{N/4}^{-1} \cdots E_2^{-1},$$

where E_k is a block diagonal matrix with $k/2$ copies of $\begin{pmatrix} I_{N/(2k)} & W_{N/(2k)} \\ I_{N/(2k)} & -W_{N/(2k)} \end{pmatrix}$ on its diagonal. Defining $\mathrm{DiagSet}(M) = \{i | \mathrm{Diag}_i(M) \neq 0\}$, E_k and its inverse have the desirable property of

$$\mathrm{DiagSet}(E_k) = \mathrm{DiagSet}(E_k^{-1}) = \{0, \pm N/(2k)\},$$

which implies that $\mathrm{HomRot}_{N/(2k)}$ and $\mathrm{HomRot}_{N/2-N/(2k)}$ are sufficient to homomorphically multiply the slot vector with E_k or E_k^{-1}. To reduce the number of levels spent during CoeffsToSlots/SlotsToCoeffs, $S_{N/2}$ and $S_{N/2}^{-1}$ are usually represented as the product of 3 to 4 matrices, each one a product of several E_ks. This technique, called level collapsing [13], offers a trade-off between the running time and the number of levels consumed.

CoeffsToSlots with subring secret encapsulation. As the key insight in our improvement of CoeffsToSlots with subring secret encapsulation, if a subring secret in R' is used and $\frac{N'}{2} \mid \frac{N}{2k}$, $\mathrm{HomRot}_{\pm N/(2k)}$ can be replaced by $\mathrm{rot}_{\pm N/(2k)}$. Then, the first $\log_2(\frac{N}{N'})$ matrices $E_2^{-1}, \cdots, E_{N/N'}^{-1}$ in the homomorphic evaluation of $S_{N/2}^{-1}$ can be computed without key switching, as long as they are evaluated under a subring secret in R'. By over-raising the ciphertext modulus in ModUp, these matrices can be multiplied with the ciphertext without consuming any levels. Our modified CoeffsToSlots with subring secret encapsulation is summarized below.

1. Over-raise the ciphertext modulus by $\log_2(\frac{N}{N'})$ levels in ModUp.
2. Homomorphically multiply the slot vector with $E_2^{-1}, E_4^{-1}, \cdots, E_{N/N'}^{-1}$.
3. Key-switch from the subring secret to the regular secret.
4. Apply homomorphic multiplication by $E_{N/2}^{-1} \cdots E_{2N/N'}^{-1}$ with the level collasping technique.

For the typical parameters of a four-level CoeffsToSlots, our modified version can use one fewer level without affecting the granularity of level collapsing in Step 4, meaning one more remaining level after bootstrapping. The modified CoeffsToSlots also has a shorter running time because the homomorphic multiplication by the first collapsed matrix in a regular CoeffsToSlots is replaced with key-switching-free matrix multiplications in Step 1. The rotation keys for Step 1 can potentially be omitted during key generation, which can reduce the size of the public key. Additionally, the linear transformations in Step 2 have a smaller modulus switching noise, since the modulus switching noise follows the same distribution as I, the overflow part in ModUp.

SlotsToCoeffs. In ModUp-first bootstrapping, the SlotsToCoeffs is unaffected by the subring secret encapsulation because the regular secret is used in this step. For StC-first bootstrapping, however, SlotsToCoeffs benefits from a subring secret as CoeffsToSlots does. Taking the commonly used three-level SlotsToCoeffs as an example, we can modify it as follows.

1. Apply the first two homomorphic matrix multiplications.
2. Key-switch to a subring secret.
3. Apply the last homomorphic matrix multiplication.
4. (Optional) Key-switch to another subring secret in a smaller subring.

Compared with regular SlotsToCoeffs, the modified version has a shorter running time because the last matrix multiplication has hoisted key-switchings. Suppose the last matrix in SlotsToCoeffs is $H = E_2 \cdot E_4 \cdots E_k$ for $k > N/N'$ (which is usually the case for three-level SlotsToCoeffs), we have $\mathrm{DiagSet}(H) = \{\frac{N}{2k} \cdot i \mid i \in [k]\}$. The homomorphic linear transformation by H on a ciphertext c can be computed with hoisted key-switchings as

$$\begin{aligned}
&\sum_{i=0}^{k-1} \mathrm{Diag}_{\frac{N}{2k} \cdot i}(M) \cdot \mathrm{HomRot}_{\frac{N}{2k} \cdot i}(c) \\
&= \sum_{v=0}^{kN'/N-1} \mathrm{KeySwitch}\Big(\sum_{u=0}^{N/N'-1} \mathrm{Diag}_{\frac{N}{2k} \cdot i} \cdot \mathrm{rot}_{\frac{N}{2k} \cdot i}(c), \mathrm{KSK}_{\mathrm{rot}_{\frac{N}{2k} \cdot v}(s') \to s'} \Big),
\end{aligned}$$

where $i = k\frac{N'}{N}u + v$. In addition to a shorter running time, subring rotation keys for SlotsToCoeffs can be generated with $a \leftarrow R'_q$ and $e \leftarrow \chi'_e$ to reduce noise growth for constant multiplication and key switchings. This does not harm the security of the scheme because the encrypted value $\mathrm{rot}_i(s')$ lies in R', unlike the key switching key from s to s' used before ModUp. Finally, we remark that the improvement in SlotsToCoeffs is independent of the secret used during ModUp. Therefore, the improved SlotsToCoeffs still works even if Step 4 uses the sparse secret encapsulation.

Sparsely packed ciphertexts. When $N^*/2$ complex values are packed into a plaintext polynomial with $N^* \mid N$ and $N^* < N$, the ciphertext/plaintext is called sparsely packed [17]. The advantage of sparsely packed ciphertext is that EvalMod needs to be executed only once instead of twice, leading to a shorter bootstrapping latency. With a sparsely packed ciphertext, CoeffsToSlots and SlotsToCoeffs involve homomorphic multiplication by $S^{-1}_{N^*/2}$ or $S_{N^*/2}$. These matrices are factored into $\log_2(N^*/2)$ matrices, where only $\max(0, \log_2(N^*/N'))$ factors can be homomorphically evaluated without key switching under a subring secret in R'. Thus, the advantage of using a subring secret in CoeffsToSlots/SlotsToCoeffs weakens as the packing becomes sparser.

For sparsely packed slots, SubSum is performed before CoeffsToSlots to evaluate Tr_{R/R^*} on the ciphertext. Therefore, CoeffsToSlots can be performed under a subring secret only if SubSum can be so. If $N' \leq N^*$, every automorphism that fixes R^* also fixes R', meaning that SubSum is key-switching-free and CoeffsToSlots can benefit from the subring secret. Otherwise, $\mathrm{Tr}_{R/R^*} = \mathrm{Tr}_{R'/R^*} \circ \mathrm{Tr}_{R/R'}$. In this case, $\mathrm{Tr}_{R/R'}$ can be computed without key-switchings, while Tr_{R'/R^*} and CoeffsToSlots need to be evaluated under the regular dense secret.

Analyzing the reduction in running time and evaluation key size.

Running time reduction. Assume CtS collapses k consecutive FFT levels, and let D be the key-switching gadget length. The first $\log(N/N')/k$ matrices in CtS each costs about $2^{k/2}D$ NTT/iNTTs and $2^{k/2}(2D + 2^{k/2})$ ring multiplications with double hoisting [9]. Under the subring secret, each matrix costs 2 NTT/iNTTs and 2 ring multiplications. Thus, the reduction in computational cost is approximately $\log(N/N')/k \cdot 2^{k/2}D - 2\log(N/N')$ NTT/iNTTs and $\log(N/N')/k \cdot 2^{k/2}(2D + 2^{k/2}) - 2\log(N/N')$ ring multiplications.

Evaluation key size reduction. We assume both CtS and StC collapse k consecutive FFT levels and use the same auxiliary modulus. In standard bootstrapping, high-level evaluation keys for CtS can be reused for low-level computation in StC. With a subring secret, the $2^{k/2+1} - 2$ evaluation keys required by the i-th CtS matrix (for $0 \leq i < \log(N/N')/k$) can be generated under $i + 2$ ciphertext primes, instead of total levels in StC+EvalMod+CtS $+ 1 - i$ primes in the standard approach. Thus, the reduction in evaluation key size is (total levels in StC+EvalMod+CtS $\cdot \log(N/N')/k - (\log(N/N')/k)^2) \cdot (2^{k/2+1} - 2)N$ word-size integers.

4 Subring Secret Encapsulation for BGV/BFV

The subring secret encapsulation can also improve the efficiency of BGV/BFV bootstrapping. However, BGV/BFV bootstrapping deviates from that of CKKS in homomorphic rounding and linear transformations, leading to distinct technical workflows and optimizations.

4.1 Digit Removal in BGV/BFV

Existing methods. Each input ciphertext for digit removal stores $p^{e-r}m_i + I_i \in \mathbb{Z}_{p^e}$ in its slots, where the noise I_i satisfies $|I_i| < K < \frac{p^{e-r}}{2}$. The aim of digit removal is to homomorphically remove the lowest $e - r$ noisy $p-$ary digits to recover $m_i \in \mathbb{Z}_{p^r}$ in the slots [33].

The digit removal makes extensive use of the lifting polynomial $F_k(X) \in \mathbb{Z}[X]$ [33] and the digit extraction polynomial $G_k(x) \in \mathbb{Z}[X]$ [14]. $F_k(X)$ has a degree of p and satisfies $F_k(x) \equiv [x]_p \bmod p^{j+2}$ for $x \equiv [x]_p \bmod p^{j+1}$ and $j \leq k$. $G_k(X)$ has a degree of $(p-1)(e-1)+1$ and satisfies $G_k(x) \equiv [x]_p \bmod p^k$. Let $w = p^{e-r}m_i + I_i$, the digit removal homomorphically calculates $w_{i,j}$ defined as follows and output $(w - \sum_{i=0}^{e-r-1} w_{i,e-i-1} \cdot p^i)/p^{e-r}$.

$$
\begin{aligned}
w_{0,0} &:= w \\
w_{i,0} &:= (w - \sum_{j=0}^{i-1} w_{j,i-j} \cdot p^j)/p^i && i = 1, 2, ..., e-r-1 \\
w_{i,j} &:= F_{j-1}(w_{i,j-1}) && i = 0, 1, ..., e-r-1, j = 1, 2, ..., e-r-i-1 \\
w_{i,e-i-1} &:= G_{e-i}(w_{i,0}) && i = 0, 1, ..., e-r-1
\end{aligned}
$$

In total, the digit removal algorithm needs to evaluate $F_k(X)$ $(e-r)(e-r-1)/2$ times and $G_k(X)$ $e-r$ times, accounting for $(e-r)(e-r+\sqrt{2(e+r)})\sqrt{p}$ ciphertext-ciphertext multiplications [28].

For moderately large values of p, $\lceil \log_p(2K-1) \rceil \leq 2$ and the evaluations of $F_k(X)$ and $G_k(X)$ can be accelerated with null polynomials [27,44]. Specifically, if $p \leq 2K-1 < p^2$, the polynomial degrees to calculate $w_{0,1} = F_1(w_{0,0})$, $w_{0,e-1} = G_e(w_{0,0})$, and $w_{1,e-2} = G_{e-1}(w_{1,0})$ can be decreased below $2K-1$, $\lceil \frac{e}{e-r} \rceil (2K-1)$, and $\lceil \frac{e-1}{e-r-1} \rceil (2\lceil \frac{K-1}{p} \rceil - 1)$. If $2K-1 < p$, the polynomial degree of $w_{0,e-1} = G_e(w_{0,0})$ can be reduced below $\lceil \frac{e}{e-r} \rceil (2K-1)$. The total number of ciphertext-ciphertext multiplications is $O(\sqrt{eK/(e-r)})$ in this case.

Using a subring secret. Recall that K is proportional to $\sqrt{h}$ and subring secret encapsulation reduces h from $\frac{2}{3}N$ to $\frac{2}{3}N'$ for uniform ternary secrets. Thus, with a large p, the subring secret encapsulation helps to decrease K from $c\sqrt{N}$ to $c\sqrt{N'}$ for some constant c, therefore lowering the complexity of digit removal from $O(\sqrt{e/(e-r)}\sqrt[4]{N})$ to $O(\sqrt{e/(e-r)}\sqrt[4]{N'})$. With a typical choice of $N = 2^{16}$ and $N' = 2^{12}$, this implies a 2x speedup in digit removal.

4.2 BGV/BFV Linear Transformations with Subring Secret

Existing methods. In BGV/BFV, the automorphism groups G and G' are generated by $\mathrm{rot}_{i,j}$ and σ, rather than by rot_i and conj in CKKS. Homomorphic linear transformations in BGV/BFV can be categorized as E-linear and $\mathbb{Z}_{p^r}$-linear transformations [32].

A one-dimensional E-linear transformation along the i-th dimension corresponds to multiplying each hypercolumn along dimension i by a matrix in $E^{l_i \times l_i}$, where the matrix for each hypercolumn can be different. Thus, such a transformation can be represented as $M = (M_j)_{0 \leq j < l/l_i}$ with $M_j \in E^{l_i \times l_i}$. We abuse the notation and let $\mathrm{diag}_u(M)$ be the R_{p^r} element encoding $(\mathrm{diag}_u(M_j))_{0 \leq j < l/l_i}$ in its slots. Let L_M be the E-linear transformation along dimension i that corresponds to $M \in (E^{l_i \times l_i})^{l/l_i}$. L_M can be homomorphically evaluated on $c = \mathrm{Enc}_s(\mathrm{Ecd}(v))$ by

$$\sum_{u=0}^{l_i-1} \mathrm{diag}_u(M) \cdot \mathrm{HomRot}_{i,u}(c) \in \mathrm{Enc}_s(\mathrm{Ecd}(L_M(v))).$$

The difference between a one-dimensional $\mathbb{Z}_{p^r}$-linear transformations along the i-th dimension and a E-linear one is that the former replaces each E-entry of $E^{l_i \times l_i}$ in the latter by a $\mathbb{Z}_{p^r}$-linear transformation on E. Each linear transformation L on E can be represented as a linearized polynomial $L(x) = \sum_{w=0}^{d-1} \kappa_w \cdot \sigma^w(x)$, $\kappa_w \in E$. For a $\mathbb{Z}_{p^r}$-linear transformation L_M, we let $\mathrm{diag}_{u,w}(M)$ be the R_{p^r} polynomial that encodes the coefficients of σ^w in the linearized polynomials of $\mathrm{diag}_u(M)$. The evaluation of L_M on c is achieved by

$$\sum_{u=0}^{l_i-1} \sum_{w=0}^{d-1} \mathrm{diag}_{u,w}(M) \cdot \mathrm{HomFrob}^w(\mathrm{HomRot}_{i,u}(c)).$$

Similarly to the linear transformations in CKKS, the linear transformations can be computed in the BSGS manner with a giant step size g by writing $u = jg + k$. We omit the BSGS equations as they can be derived similarly to the CKKS case.

Using a subring secret. We analyze how $\mathrm{HomRot}_{i,u}$ and σ act on the subring R'. Recall that $\mathrm{HomRot}_{i,u}$ is a weighted sum of $\theta_{g_i^u}$ and $\theta_{g_i^{u-l_i}}$. Thus, it suffices to check the structure of the plaintext automorphisms in R'_{p^r}, where the generators used in plaintext encoding are still g_i. Lemma 3 helps to characterize the plaintext space in R'_{p^r}.

Lemma 3. *[26] Let $N \geq 2$ be a power of two and consider an odd prime p. If $p = 4r \cdot k + 1$ with r a power of two and k odd, $l_1 = \min(r, N/2)$. If $p = 4r \cdot k - 1$ with r a power of two and k odd, $l_1 = \min(2r, N/2)$.*

Define $d_{\min} = \min(d)_{N \geq 2}$ and $l_{\max} = \max(l_1)_{N \geq 2}$. $d_{\min} = 1$ and $l_{\max} = r$ if $p \equiv 1 \bmod 4$, while $d_{\min} = 2$ and $l_{\max} = 2r$ if $p \equiv 3 \bmod 4$. The plaintext structure of R'_{p^r} can be categorized into three cases as in Lemma 4, depending on whether $\ker f$ contains only rotations, only Frobenius automorphisms, or both.

Lemma 4. *With the notations defined above,* $\ker f$ *falls into three categories.*

1. $l'_1 = l_{\max}$*, then* $l_1 = l_{\max}$ *and* $d = \frac{N}{N'} d'$.
   $\ker f = \langle \sigma^{d'} \rangle$.
2. $l'_1 \leq \frac{N'}{N} l_{\max}$*, then* $l_1 = \frac{N}{N'} l'_1 \leq l_{\max}$. $d' = d = d_{\min}$.
   $\ker f = \langle \theta_{5^{l'_1}} \rangle$.
3. $\frac{N'}{N} l_{\max} < l'_1 < l_{\max}$*, then* $l_1 = l_{\max}$, $d = \frac{N l'_1}{N' l_{\max}} d' > d' = d_{\min}$.
   $\ker f = \langle \theta_{5^{l'_1}}, \sigma^{d_{\min}} \rangle$

Proof. In cases 1 and 3, $\sigma^{d'} \in \ker f$ from the definition of d'. In cases 2 and 3, the first dimension in R'_{p^r} is good because $d' = d_{\min}$, so $\theta_{5^{l'_1}}$ fixes the slot vector. Thus, $\ker f$ contains a subgroup generated by one or both of $\sigma^{d'}$ and $\theta_{5^{l'_1}}$. The other direction of inclusion follows from $\#\ker f = N/N'$. Also note that when $p \equiv 1 \bmod 4$, θ_{-1}, the rotation in the second dimension, is not in $\ker f$. □

CoeffsToSlots/SlotsToCoeffs for densely packed ciphertexts. Recall that CoeffsToSlots should be partly evaluated under a subring secret to hoist the key switchings corresponding to $\ker f$. Thus, we cannot adopt the usual inner product method to increase the homomorphic capacity [28], which switches the ciphertext secret to the regular secret. Instead, we use modulus switching for BFV [4] or ModUp for BGV [44] to obtain a ciphertext with large homomorphic capacity under the subring secret.

As in CKKS, the linear transformations in CoeffsToSlots/SlotsToCoeffs factor into multiple subtransformations. Two different factorization methods have been proposed by Geelen [26] and Ma et al. [45]. As summarized in [45], the

preference for factorization depends on the type of bootstrapping and the value of p and is given in Table 2. Only the preferred factorization method will be discussed in the following. We assume the slots are unpacked after CoeffsToSlots using the method from [26].

Table 2. Comparison of the factorization methods in different scenarios.

	$p \equiv 1 \bmod 4$	$p \equiv 3 \bmod 4$
General bootstrapping	Ma et al.'s	Geelen's for small d Ma et al.'s for large d
Slim bootstrapping	Both are equal	Geelen's

For general bootstrapping with $p \equiv 1 \bmod 4$, Ma et al.'s method factors CoeffsToSlots into

$$N_{\log_2(l_1)+1} \cdot N_{\log_2(l_1)} \cdots N_1 \cdot \mathrm{Eval}^{-1},$$

where Eval is a slotwise $\mathbb{Z}_{p^e}$-linear transformation. For $1 \leq i \leq \log_2(l_1)$, N_i is a E-linear transformations in the first dimension with $\mathrm{DiagSet}(N_i) = \{0, \pm 2^{-i} l_1\}$, while $N_{\log_2(l_1)+1}$ is in the second dimension with $\mathrm{DiagSet}(N_{\log_2(l_1)+1}) = \{0, 1\}$.

For $p \equiv 3 \bmod 4$, let $E'' = \mathbb{Z}_{p^e}[\zeta_4]$, Geelen's method factors CoeffsToSlots into

$$M^{-1} \cdot M_{\log_2(l_1)+1} \cdot M_{\log_2(l_1)} \cdots M_1 \cdot M,$$

where M is a slotwise E''-linear transformation and can be computed by a weighted sum of $(\sigma^{2i})_{0 \leq i < d/2}$. M_i has the same type as N_i and $\mathrm{DiagSet}(M_i) = \mathrm{DiagSet}(N_i)$.

Using a subring secret. With a subring secret in R', homomorphic multiplications by N_i and M_i do not require key switching if $i \leq \log_2(l_1/l_1')$. The condition that Eval^{-1} and M are key-switching-free is $d' = d_{\min}$[4]. Thus, $N_{\log_2(l_1/l_1')} \cdots N_1 \cdot \mathrm{Eval}^{-1}$ or $M_{\log_2(l_1/l_1')} \cdots M_1 \cdot M$ can be computed without key switching in cases 2 and 3, or in case 1 with $d' = d_{\min}$. Overraising the ciphertext modulus helps to remove the noise growth caused by these key-switching-free linear transformations, similar to the discussion in CKKS. Even if $d' > d_{\min}$ in case 1, Eval^{-1} and M could still benefit from the subring secret if the Frobenius automorphism keys are generated under the regular secret. E.g., we could compute Eval^{-1} as $\mathrm{Eval}^{-1}(c) = \sum_{j=0}^{d/d'-1} \mathrm{KeySwitch}(\sum_{k=0}^{d'-1} \kappa_w \cdot \sigma^{jd'+k}(c), \mathrm{KSK}_{\sigma^k(s') \to s})$.

The modification to SlotsToCoeffs in StC-first bootstrapping when using a subring secret follows from the discussion of CoeffsToSlots and the discussion of SlotsToCoeffs in CKKS. Suppose SlotsToCoeffs is represented as the product of three matrices using the level collasping technique, the first two will be

[4] Ma et al.'s method for $p \equiv 3 \bmod 4$ involves the computation of Eval^{-1}, which is a weighted sum of $(\sigma^i)_{0 \leq i < d}$ and is not key-switching-free even for $d' = d_{\min} = 2$. Therefore, we exclude it from our discussion here.

evaluated under the regular secret before a key-switching to a subring secret takes place. For some $k > \log_2(l_1/l_1')$, the last matrix is either equal to $H = \mathrm{Eval} \cdot N_1^{-1} \cdot N_2^{-1} \cdots N_k^{-1}$ when $p \equiv 1 \bmod 4$, or $H = M^{-1} \cdot M_1^{-1} \cdot M_2^{-1} \cdots M_k^{-1}$. As $\mathrm{DiagSet}(H) = \{2^{-k} l_1 \cdot i \mid i \in [2^k]\}$, the homomorphic multiplication of H with a ciphertext c can be evaluated as

$$\sum_{u=0}^{2^k-1} \sum_{w=0}^{d/d_{\min}-1} \mathrm{diag}_{u \cdot 2^{-k} \cdot l_1, w \cdot d_{\min}}(H) \cdot \mathrm{HomFrob}^{w \cdot d_{\min}}(\mathrm{HomRot}_{1, u \cdot 2^{-k} \cdot l_1}(c)).$$

Let $\mathrm{diag}_{u,w} = \mathrm{diag}_{u \cdot 2^{-k} \cdot l_1, w \cdot d_{\min}}(H)$ and $v = u \cdot 2^{-k} \cdot l_1$, the key-switchings for evaluating the equation above can be hoisted under a subring secret, where the manner of hoisting depends on the structure of $\ker f$.

- For case 1 in Lemma 4, H is a one-dimensional $\mathbb{Z}_{p^r}$-linear transformation and only HomFrob can be hoisted, leading to

  $$\sum_{j=0}^{d'/d_{\min}-1} \mathrm{KeySwitch}\Big(\sum_{i=0}^{d/d'-1} \sigma^{w \cdot d_{\min}} \Big(\sum_{u=0}^{2^k-1} \mathrm{diag}_{u,w} \cdot \mathrm{HomRot}_{1,v}(c) \Big), \mathrm{KSK}_{\sigma^{j \cdot d_{\min}}(s') \to s'} \Big),$$

  where $w = id'/d_{\min} + j$.
- For case 2, Eval and M degenerates to identity transformation, turning H into a one-dimensional E-linear transformation. As $d = d_{\min}$, the first dimension is good, leading to

  $$\sum_{j=0}^{2^k \cdot l_1'/l_1 - 1} \mathrm{KeySwitch}\Big(\sum_{i=0}^{l_1/l_1'-1} \mathrm{diag}_u \cdot \mathrm{rot}_{1,v}(c), \mathrm{KSK}_{\theta_{5^{2^{-k} l_1 \cdot j}}(s') \to s'} \Big),$$

  where $u = 2^k \cdot l_1'/l_1 \cdot i + j$.
- For case 3, HomFrob is key-switching free since $\sigma^{d_{\min}} \in \ker f$. However, as the first dimension is bad, $\mathrm{rot}_{1,v}(\cdot) = \mu_{1,v} \theta_{5^v}(\cdot) + (1 - \mu_{1,v}) \theta_{5^{v-l_1}}(\cdot)$. For a regular secret, this means both $\mathrm{KSK}_{\theta_{5^v}(s) \to s}$ and $\mathrm{KSK}_{\theta_{5^{v-l_1}(s) \to s}}$ are required for key-switching. With a subring secret, however, $\theta_{5^{v-l_1}}(s') = \theta_{5^v}(s')$ due to $\theta_{5^{l_1}} = \theta_{5^{l_1'}}^{l_1/l_1'} \in \ker f$, which means only $\mathrm{KSK}_{\theta_{5^{v \bmod l_1'}}(s') \to s'}$ is required. The formula becomes

  $$\sum_{j=0}^{2^k \cdot l_1'/l_1 - 1} \mathrm{KeySwitch}\Big(\sum_{i=0}^{l_1/l_1'-1} \sum_{w=0}^{d/d_{\min}-1} \mathrm{diag}_{u, w \cdot d_{\min}} \cdot \sigma^{w \cdot d_{\min}}(\mathrm{rot}_{1,v}(c)), \mathrm{KSK}_{\theta_{5^{2^{-k} l_1 \cdot j}}(s') \to s'} \Big),$$

  where $u = 2^k \cdot l_1'/l_1 \cdot i + j$.

Sparsely packed ciphertexts. With sparsely packed slots, Eval and M (and their inverses) in CoeffsToSlots/SlotsToCoeffs can be omitted. SlotsToCoeffs turns into a E-linear transformation and still benefits from the subring secret in

cases 2 and 3 in Lemma 4. For CoeffsToSlots, extra homomorphic trace maps are required before and after it to clear the extra coefficients. For $p \equiv 1 \bmod 4$, an extra Tr_{R/R^*} is needed before CoeffsToSlots. For $p \equiv 3 \bmod 4$, Tr_{R/R^*} and $\mathrm{Tr}_{E''/\mathbb{Z}_{p^e}}$ are needed before and after CoeffsToSlots. CoeffsToSlots can only benefit from the subring secret if the trace map before it, Tr_{R/R^*}, does not require key switching, corresponding to $R' \leq R^*$. In this case, the matrices M_i or N_i in CoeffsToSlots with $i \leq \log_2(l_1/l_1')$ can still be evaluated under the subring secret with little time and no capacity consumption. Otherwise, we will end with a ciphertext under the regular key s after the homomorphic trace evaluation, so the CoeffsToSlots can no longer benefit from the subring secret encapsulation. However, the trace itself can still utilize the subring secret, as $\mathrm{Tr}_{R/R^*} = \mathrm{Tr}_{R'/R^*} \circ \mathrm{Tr}_{R/R'}$ and the latter trace is key-switching-free.

Non-power-of-two Cyclotomic Rings. Although our discussion is limited to power-of-two cyclotomic rings, a subring secret benefits BFV/BGV bootstrapping in a non-power-of-two case as well. The improvement in digit removal follows from the secret's lower Hamming weight. For linear transformations, we calculate the orders of the hypercube generators and Frobenius automorphism in the subring to decide which key-switchings can be hoisted.

5 Implementation

5.1 CKKS Bootstrapping.

We implemented CKKS bootstrapping with subring secret encapsulation in Lattigo [2] with commit ID b98d8915[5]. All experiments are conducted on an Intel(R) Xeon(R) Gold 6248R CPU @ 3.00GHz CPU running Fedora 38.

Parameters setting. The ciphertext is assumed to be densely packed with a ring dimension of $N = 2^{16}$. The regular secret follows the uniform ternary distribution. χ_e is the discrete Gaussian distribution with a standard deviation of 3.19. We use ModUp-first bootstrapping to be consistent with the benchmarks in existing works. CoeffsToSlots is factored into four matrices, while SlotsToCoeffs is factored into three. We also scale the message up before ModUp so that $\log(q_0/(\Delta \text{ after scale})) = 8$ as in the implementation of [9] and [8]. All parameters satisfy 128 bits of security according to Table 1.

Table 3. Parameters for subring secret encapsulation before ModUp. Q' and P' are the ciphertext modulus and auxiliary modulus for subring key switching, respectively.

N'	$\log Q'$	$\log P'$
4096	45	61

[5] Implementation available at `https://github.com/msh086/subring_bts`

The parameter sets used are listed in Table 4, with the subring secret encapsulation parameters detailed in Table 3. I_1 and I_2 exhibit failure probabilities similar to those of the parameters in [9]. I_1 has an after-bootstrap homomorphic capacity comparable to [9], while I_2 is selected to maximize the number of usable levels after bootstrapping. The differences between II_1 and II_2 and the parameters in [8] mirror those between I_1 and I_2 and the parameters in [9].

Table 4. Parameter sets for CKKS bootstrapping with different secret encapsulation approaches. The failure probability of bootstrapping is $\kappa = \Pr[||I||_\infty > K]$. L is the number of remaining levels after bootstrapping.

	[9]	[8]	[10]	I_1	I_2	II_1	II_2	III_1	III_2	IV_1	IV_2
Secret encapsulation	None		Sparse	Subring							
$\log(QP)$	1731	1734	1737	1555	1730	1605	1745	1606	1746	1606	1746
L	9	10	14	10	15	10	14	10	14	10	14
$\log(q_0)$	50	45	50	45		45		45		45	
$\log \Delta$	40	35	40	35		35		35		35	
$\log \Delta_{\mathrm{StC}}$	28	30	39	33		32		31		30	
$\log \Delta_{\mathrm{CtS}}$	53	56	56	52		50		51		52	
$\log \Delta_{\mathrm{bts}}$	60	60	60	60		60		60		60	
$\log(P)$	305	305	305	305		305		305		305	
d	255	255	31	127		127		127		255	
r	4	4	3	3		4		4		3	
K	383	514	16	102		135		166		228	
$\log \kappa$	-15.0	-37.9	-138.7	-15.8		-37.8		-64.0		-132.0	

To ensure a fair comparison, we made several modifications to the parameter sets in existing works. The secret from [9] had a fixed Hamming weight of 32768 and is replaced with a uniform ternary secret. The number of bits in the maximum ciphertext modulus is also reduced below 1747 to ensure a 128-bit security. The auxiliary modulus for key switching, denoted as P, is set to the product of five 61-bit primes. Finally, the value of K is adjusted to meet the failure probability requirements in [9] and [8], where the failure probability is estimated using the method described in Section 3.1.

Experimental results. Following [9,8], the precision of the bootstrapping is defined as $-\log \epsilon$, where ϵ is the average L_1 norm between the slot vector before and after the bootstrapping. The bootstrapping performance is measured with the bootstrapping throughput [9], which is defined as

$$\text{Throughput} = \frac{N/2 \times \log(\epsilon^{-1}) \times L \times \log(\Delta)}{\text{Running time}}.$$

The experimental results are presented in Table 5.

Table 5. Experiment results for ModUp-first CKKS bootstrapping with different secret encapsulation approaches. $-\log\epsilon$ represents the bootstrapping precision where ϵ is the average L1 norm of the bootstrapping error. L is the number of available levels after bootstrapping. $\kappa = \Pr[||I||_\infty > K]$ is the bootstrapping failure probability.

	[9]	[8]	[10]	I_1	I_2	II_1	II_2	III_1	III_2	IV_1	IV_2
	Secret encapsulation										
	None		Sparse	Subring							
	Time (s)										
ModUp	0.12	0.13	1.44	0.14	0.16	0.13	0.16	0.14	0.14	0.13	0.17
CtS	12.85	14.16	14.01	11.57	15.24	12.15	15.26	12.18	15.23	12.16	15.23
EvalMod	22.37	25.40	10.44	18.27	24.67	19.50	25.25	19.54	25.21	23.25	30.19
StC	4.50	4.93	7.30	4.96	7.73	4.92	7.38	4.94	7.33	4.92	7.39
Total	39.84	44.62	33.27	34.93	47.78	36.71	48.04	36.80	47.92	40.46	52.98
	$-\log\epsilon$										
	16.53	15.80	22.88	19.50	19.49	18.36	18.35	17.98	17.97	17.11	17.10
	L										
	9	10	14	10	15	10	14	10	14	10	14
	$\log\kappa$										
	-15.0	-37.9	-138.7	-15.8		-37.8		-64.0		-132.0	
	Throughput (Mbps)										
	4.67	3.87	12.03	6.11	6.70	5.47	5.85	5.34	5.74	4.63	4.94

Result analysis. I_1 runs faster than [9] (and II_1 faster than[8]) because the use of the subring secret reduces the computational complexity of CoeffsToSlots and EvalMod. Their bootstrapping precision is also higher by 2.5∼3 bits due to the reduced approximation range of the modular reduction function. Also, I_2 and II_2 have more levels left after bootstrapping than in the existing works. The reasons are that the first matrix in CoeffsToSlots becomes level-free under a subring secret, and EvalMod requires a smaller d or r, saving another one or two levels. Since each RNS prime in CoeffsToSlots and EvalMod takes more bits than the scaling factor Δ, the saved $2 \sim 3$ levels translate to $3 \sim 4$ more levels with prime size $\approx \Delta$. The final extra level in I_2 and II_2 comes from decreased Δ or Δ_{CtS}. Combining the improvements in running time, bootstrapping precision, and remaining levels, I_2 has a throughput 46% higher than [9], and II_2 51% higher than [8].

Notably, more available levels after bootstrapping mean the bootstrapping is performed at a higher level and is therefore slower. Since the bit length of Δ is far from 60, the maximum length for RNS primes in Lattigo, Grafting might be a solution to this problem. It factors the ciphertext modulus into nearly word-size primes to reduce the number of RNS primes, accelerating homomorphic computation [15].

StC-first bootstapping. Recall that StC is performed at a lower level in StC-first bootstrapping, leading to a lower running time compared to ModUp-first bootstrapping. According to our tests, the running time of StC is 1.19∼1.32 seconds with none or sparse secret encapsulation, and can be lowered to 1.02∼1.10 seconds using a subring secret with $N'_{\mathrm{StC}} = 8192$.

5.2 BGV/BFV Bootstrapping.

We implemented subring secret encapsulation on Ma et al.'s implementation [45], which is based on HElib [1] with commit ID 3e337a6.[6] The experiments are conducted on the same machine as in the CKKS experiments. The scheme parameters are summarized in Table 6.

Table 6. Parameters for slim BGV/BFV bootstrapping with sparsely packed ciphertexts. The main secret is sampled from the uniform ternary distribution.

N	p	r	l	d	e	$\log(Q)$	$\log(P)$	N'	$\log(Q')$	$\log(P')$
65536	65537	1	32768	2	2	1390	341	4096	53	21

Table 7. Experiment results for slim BGV/BFV bootstrapping with different secret encapsulation approaches. $h = 32$ for the sparse secret.

Secret Encapsulation		None	Sparse	Subring
$\log \kappa$		−130.5	−138.7	−130.5
K		861	16	227
Time (s)	SlotsToCoeffs	7.67	7.68	7.59
	Digit removal	170.3	17.02	75.36
	CoeffsToSlots	35.90	36.22	30.96
	Total	214.98	62.16	114.67
Capacity (bit)	SlotsToCoeffs	80	80	80
	Digit removal	634	295	510
	CoeffsToSlots	175	181	131
	Remaining	452	771	600
Throughput (Mbps)		1.051	6.202	2.616

The benchmark results with subring secret encapsulation, sparse secret encapsulation, and no secret encapsulation are presented in Table 7. The boot-

[6] As a proof-of-concept implementation, we only considered the case of $p \equiv 1 \bmod 4$ and did not implement the optimizations on SlotsToCoeffs.

strapping throughput is defined similarly to CKKS as

$$\text{Throughput} = \frac{l \times \log(p^r) \times (\text{Homomorphic capacity})}{\text{Running time}}.$$

The polynomial evaluation method in the state-of-the-art dense-key BGV/BFV bootstrapping [48] does not apply to this parameter set because their approach is only effective for $d > 2$. Thus, the benchmark baseline is taken as the implementation from [45] without secret encapsulation.

Result analysis. Compared to secret encapsulation or the dense key bootstrapping without secret encapsulation, subring secret encapsulation reduced the computation time and the capacity consumption of CoeffsToSlots as expected. Also, the value of K is decreased by $\sqrt{N/N'} = 4$ times in the subring secret case, leading to a digit removal that is 2.28x faster than the dense secret case and saves 100 bits of capacity at the same time. Combined, the bootstrapping throughput with a subring secret is 2.48x that of a regular dense secret.

Acknowledgements

This work is supported by the National Cryptologic Science Fund of China (2025NCSF02041), the Scientific Research Innovation Capability Support Project for Young Faculty (ZYGXQNJSKYCXNLZCXM-P4), the National Key R&D Program of China (2020YFA0309705), the Key R&D Program of Shandong Province (2024ZLGX05), and Tsinghua University Dushi Program.

References

1. HElib. Online: `https://github.com/homenc/HElib` (Jul 2023), iBM Corp
2. Lattigo v6. Online: `https://github.com/tuneinsight/lattigo` (Aug 2024), ePFL-LDS, Tune Insight SA
3. Albrecht, M., Chase, M., Chen, H., Ding, J., Goldwasser, S., Gorbunov, S., Halevi, S., Hoffstein, J., Laine, K., Lauter, K., Lokam, S., Micciancio, D., Moody, D., Morrison, T., Sahai, A., Vaikuntanathan, V.: Homomorphic Encryption Standard, pp. 31–62. Springer International Publishing, Cham (2021). `https://doi.org/10.1007/978-3-030-77287-1_2`
4. Alperin-Sheriff, J., Peikert, C.: Practical Bootstrapping in Quasilinear Time. In: Canetti, R., Garay, J.A. (eds.) Advances in Cryptology – CRYPTO 2013. pp. 1–20. Springer Berlin Heidelberg, Berlin, Heidelberg (2013). `https://doi.org/10.1007/978-3-642-40041-4_1`
5. Bae, Y., Cheon, J.H., Cho, W., Kim, J., Kim, T.: META-BTS: Bootstrapping Precision Beyond the Limit. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. p. 223–234. CCS '22, Association for Computing Machinery, New York, NY, USA (2022). `https://doi.org/10.1145/3548606.3560696`

6. Bae, Y., Cheon, J.H., Kim, J., Park, J.H., Stehlé, D.: HERMES: Efficient Ring Packing Using MLWE Ciphertexts and Application to Transciphering. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology – CRYPTO 2023. pp. 37–69. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-38551-3_2
7. Bajard, J.C., Eynard, J., Hasan, M.A., Zucca, V.: A Full RNS Variant of FV Like Somewhat Homomorphic Encryption Schemes. In: Avanzi, R., Heys, H. (eds.) Selected Areas in Cryptography – SAC 2016. pp. 423–442. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-69453-5_23
8. Bossuat, J.P., Cammarota, R., Chillotti, I., Curtis, B.R., Dai, W., Gong, H., Hales, E., Kim, D., Kumara, B., Lee, C., Lu, X., Maple, C., Pedrouzo-Ulloa, A., Player, R., Polyakov, Y., Lopez, L.A.R., Song, Y., Yhee, D.: Security Guidelines for Implementing Homomorphic Encryption. IACR Communications in Cryptology **1**(4) (2025). https://doi.org/10.62056/anxra69p1
9. Bossuat, J.P., Mouchet, C., Troncoso-Pastoriza, J., Hubaux, J.P.: Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021. pp. 587–617. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-77870-5_21
10. Bossuat, J.P., Troncoso-Pastoriza, J., Hubaux, J.P.: Bootstrapping for Approximate Homomorphic Encryption with Negligible Failure-Probability by Using Sparse-Secret Encapsulation. In: Ateniese, G., Venturi, D. (eds.) Applied Cryptography and Network Security. pp. 521–541. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-09234-3_26
11. Brakerski, Z.: Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) Advances in Cryptology – CRYPTO 2012. pp. 868–886. Springer Berlin Heidelberg, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-32009-5_50
12. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) Fully Homomorphic Encryption without Bootstrapping. ACM Trans. Comput. Theory **6**(3) (Jul 2014). https://doi.org/10.1145/2633600
13. Chen, H., Chillotti, I., Song, Y.: Improved Bootstrapping for Approximate Homomorphic Encryption. In: Ishai, Y., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2019. pp. 34–54. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-17656-3_2
14. Chen, H., Han, K.: Homomorphic Lower Digits Removal and Improved FHE Bootstrapping. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018. pp. 315–337. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-78381-9_12
15. Cheon, J.H., Choe, H., Kang, M., Kim, J., Kim, S., Mono, J., Noh, T.: Grafting: Decoupled Scale Factors and Modulus in RNS-CKKS. Cryptology ePrint Archive, Paper 2024/1014 (2024), https://eprint.iacr.org/2024/1014
16. Cheon, J.H., Choe, H., Passelègue, A., Stehlé, D., Suvanto, E.: Attacks Against the IND-CPA$^{\mathrm{D}}$ Security of Exact FHE Schemes. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 2505–2519. CCS '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3658644.3690341
17. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for Approximate Homomorphic Encryption. In: Nielsen, J.B., Rijmen, V. (eds.) Advances in Cryptology – EUROCRYPT 2018. pp. 360–384. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-78381-9_14

18. Cheon, J.H., Hhan, M., Hong, S., Son, Y.: A Hybrid of Dual and Meet-in-the-Middle Attack on Sparse and Ternary Secret LWE. IEEE Access **7**, 89497–89506 (2019). https://doi.org/10.1109/ACCESS.2019.2925425
19. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic Encryption for Arithmetic of Approximate Numbers. In: Takagi, T., Peyrin, T. (eds.) Advances in Cryptology – ASIACRYPT 2017. pp. 409–437. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-70694-8_15
20. Cheon, J.H., Son, Y., Yhee, D.: Practical FHE parameters against lattice attacks. Journal of the Korean Mathematical Society **59**(1), 35–51 (01 2022). https://doi.org/10.4134/JKMS.j200650
21. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast Fully Homomorphic Encryption Over the Torus. Journal of Cryptology **33**(1), 34–91 (Jan 2020). https://doi.org/10.1007/s00145-019-09319-x
22. Curtis, B.R., Player, R.: On the Feasibility and Impact of Standardising Sparse-secret LWE Parameter Sets for Homomorphic Encryption. In: Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography. p. 1–10. WAHC'19, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3338469.3358940
23. Ducas, L., Micciancio, D.: FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology – EUROCRYPT 2015. pp. 617–640. Springer Berlin Heidelberg, Berlin, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46800-5_24
24. Elliott, D., Paget, D., Phillips, G., Taylor, P.: Error of truncated Chebyshev series and other near minimax polynomial approximations. Journal of Approximation Theory **50**(1), 49–57 (1987). https://doi.org/10.1016/0021-9045(87)90065-7
25. Fan, J., Vercauteren, F.: Somewhat Practical Fully Homomorphic Encryption. Cryptology ePrint Archive, Paper 2012/144 (2012), https://eprint.iacr.org/2012/144
26. Geelen, R.: Revisiting the Slot-to-Coefficient Transformation for BGV and BFV. IACR Communications in Cryptology **1**(3) (2024). https://doi.org/10.62056/a01zogy4e-
27. Geelen, R., Iliashenko, I., Kang, J., Vercauteren, F.: On Polynomial Functions Modulo p^e and Faster Bootstrapping for Homomorphic Encryption. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology – EUROCRYPT 2023. pp. 257–286. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-30620-4_9
28. Geelen, R., Vercauteren, F.: Bootstrapping for BGV and BFV Revisited. Journal of Cryptology **36**(2), 12 (Mar 2023). https://doi.org/10.1007/s00145-023-09454-6
29. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing. p. 169–178. STOC '09, Association for Computing Machinery, New York, NY, USA (2009). https://doi.org/10.1145/1536414.1536440
30. Gentry, C., Halevi, S., Peikert, C., Smart, N.P.: Ring Switching in BGV-Style Homomorphic Encryption. In: Visconti, I., De Prisco, R. (eds.) Security and Cryptography for Networks. pp. 19–37. Springer Berlin Heidelberg, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-32928-9_2
31. Halevi, S., Shoup, V.: Algorithms in HElib. In: Garay, J.A., Gennaro, R. (eds.) Advances in Cryptology – CRYPTO 2014. pp. 554–571. Springer Berlin Heidelberg, Berlin, Heidelberg (2014). https://doi.org/10.1007/978-3-662-44371-2_31

32. Halevi, S., Shoup, V.: Faster Homomorphic Linear Transformations in HElib. In: Shacham, H., Boldyreva, A. (eds.) Advances in Cryptology – CRYPTO 2018. pp. 93–120. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-96884-1_4
33. Halevi, S., Shoup, V.: Bootstrapping for HElib. Journal of Cryptology **34**(1), 7 (Jan 2021). https://doi.org/10.1007/s00145-020-09368-7
34. Han, K., Hhan, M., Cheon, J.H.: Improved Homomorphic Discrete Fourier Transforms and FHE Bootstrapping. IEEE Access **7**, 57361–57370 (2019). https://doi.org/10.1109/ACCESS.2019.2913850
35. Han, K., Ki, D.: Better Bootstrapping for Approximate Homomorphic Encryption. In: Jarecki, S. (ed.) Topics in Cryptology – CT-RSA 2020. pp. 364–390. Springer International Publishing, Cham (2020). https://doi.org/10.1007/978-3-030-40186-3_16
36. Hhan, M., Kim, J., Lee, C., Son, Y.: Let's Meet Ternary Keys on Babai's Plane: A Hybrid of Lattice-reduction and Meet-LWE. Cryptology ePrint Archive, Paper 2022/1473 (2022), https://eprint.iacr.org/2022/1473
37. Hoffstein, J., Pipher, J., Silverman, J.H.: NTRU: A ring-based public key cryptosystem. In: Buhler, J.P. (ed.) Algorithmic Number Theory. pp. 267–288. Springer Berlin Heidelberg, Berlin, Heidelberg (1998). https://doi.org/10.1007/BFb0054868
38. Jutla, C.S., Manohar, N.: Sine Series Approximation of the Mod Function for Bootstrapping of Approximate HE. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022. pp. 491–520. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-06944-4_17
39. Lee, E., Lee, J., Son, Y., Wang, Y.: Improved Meet-LWE Attack via Ternary Trees. Cryptology ePrint Archive, Paper 2024/824 (2024), https://eprint.iacr.org/2024/824
40. Lee, J.W., Lee, E., Lee, Y., Kim, Y.S., No, J.S.: High-Precision Bootstrapping of RNS-CKKS Homomorphic Encryption Using Optimal Minimax Polynomial Approximation and Inverse Sine Function. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021. pp. 618–647. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-77870-5_22
41. Lee, K.H., Yoon, J.W.: Discretization Error Reduction for High Precision Torus Fully Homomorphic Encryption. In: Boldyreva, A., Kolesnikov, V. (eds.) Public-Key Cryptography – PKC 2023. pp. 33–62. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-31371-4_2
42. Lee, Y., Lee, J.W., Kim, Y.S., Kim, Y., No, J.S., Kang, H.: High-Precision Bootstrapping for Approximate Homomorphic Encryption by Error Variance Minimization. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022. pp. 551–580. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-06944-4_19
43. Lyubashevsky, V., Peikert, C., Regev, O.: On Ideal Lattices and Learning with Errors over Rings. J. ACM **60**(6) (Nov 2013). https://doi.org/10.1145/2535925
44. Ma, S., Huang, T., Wang, A., Wang, X.: Accelerating BGV Bootstrapping for Large p Using Null Polynomials over $\mathbb{Z}_{p^e}$. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024. pp. 403–432. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-58723-8_14
45. Ma, S., Huang, T., Wang, A., Wang, X.: Faster BGV Bootstrapping for Power-of-Two Cyclotomics Through Homomorphic NTT. In: Chung, K.M., Sasaki, Y. (eds.) Advances in Cryptology – ASIACRYPT 2024. pp. 143–175. Springer Nature Singapore, Singapore (2025). https://doi.org/10.1007/978-981-96-0875-1_5

46. May, A.: How to Meet Ternary LWE Keys. In: Malkin, T., Peikert, C. (eds.) Advances in Cryptology – CRYPTO 2021. pp. 701–731. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-84245-1_24
47. Nolte, N., Malhou, M., Wenger, E., Stevens, S., Li, C., Charton, F., Lauter, K.: The Cool and the Cruel: Separating Hard Parts of LWE Secrets. In: Vaudenay, S., Petit, C. (eds.) Progress in Cryptology - AFRICACRYPT 2024. pp. 428–453. Springer Nature Switzerland, Cham (2024). https://doi.org/10.1007/978-3-031-64381-1_19
48. Okada, H., Player, R., Pohmann, S.: Homomorphic Polynomial Evaluation Using Galois Structure and Applications to BFV Bootstrapping. In: Guo, J., Steinfeld, R. (eds.) Advances in Cryptology – ASIACRYPT 2023. pp. 69–100. Springer Nature Singapore, Singapore (2023). https://doi.org/10.1007/978-981-99-8736-8_3
49. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. J. ACM **56**(6) (Sep 2009). https://doi.org/10.1145/1568318.1568324